

Reviewer Pack — Additive Energy Inverse Proportionality to ACC⁰ Circuit Size...

Entry 249d7867db76 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 11:00:13 UTC

Additive Energy Inverse Proportionality to ACC⁰ Circuit Size for Explicit Functions

Entry ID: 249d7867db76 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 11:00:13 UTC

1. Conjecture

Field A (mathematical branch): Additive Combinatorics **Field B** (complexity object): ACC⁰ Circuit Size

Statement:

For an explicit function f in P with n variables, the additive energy $E(f)$ of the set derived from its truth table is inversely proportional to the minimum ACC⁰ circuit size $S(f)$, i.e., $E(f) * S(f)^\beta \geq C * n^\alpha$ for constants $\alpha, \beta, C > 0$.

Rationale (proposer's reasoning):

Additive energy captures structural regularity, which may conflict with the structured computations of ACC⁰ circuits. High energy implies low circuit size, but this inverse relationship is conjectured to hold for explicit functions in P .

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4c2004bfff29a7530

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=8.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

# Helper functions for matrix operations and energy calculation
def matrix_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    n = len(A)
    M = [A[i] + [b[i]] for i in range(n)]
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(M[j][i]) > abs(M[max_row][i]):
                max_row = j
        M[i], M[max_row] = M[max_row], M[i]
        factor = M[i][i]
        for j in range(n + 1):
            M[i][j] /= factor
        for j in range(n):
            if j != i:
                factor = M[j][i]
                for k in range(n + 1):
                    M[j][k] -= factor * M[i][k]
    return [row[-1] for row in M]

def additive_energy(truth_table):
    n = len(truth_table)
    energy = 0
    for i in range(n):
        for j in range(i+1, n):
            for k in range(j+1, n):
                for l in range(k+1, n):
```

```

        if (truth_table[i][j] == truth_table[k][l]) != (truth_table[i][k] == truth_table[i][l]):
            energy += 1

    return energy

# Function to estimate ACC0 circuit size (simplified heuristic)
def estimate_acc0_size(truth_table):
    n = len(truth_table)
    # This is a very naive heuristic; in practice, this would be much more complex
    return n * math.log2(n)

# Main function for running one trial
def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    truth_table = [[random.randint(0, 1) for _ in range(n)] for _ in range(2**n)]

    energy = additive_energy(truth_table)
    acc0_size = estimate_acc0_size(truth_table)

    if acc0_size == 0:
        return {
            "metric_name": "additive_energy",
            "metric_value": energy,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    beta = Fraction(1, 2) # Example value for beta
    alpha = Fraction(1, 3) # Example value for alpha
    C = 1.0 # Example constant

    if energy * acc0_size**beta >= C * n**alpha:
        return {
            "metric_name": "additive_energy",
            "metric_value": energy,
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": ""
        }
    else:
        return {
            "metric_name": "additive_energy",
            "metric_value": energy,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"Counterexample: n={n}, energy={energy}, acc0_size={acc0_size}"
        }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)] # Default to first

    results = []
    total_energy = 0
    count_true = 0

```

```

counterexample = ""

for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")
    results.append(trial_result)
    total_energy += trial_result["metric_value"]
    if trial_result["conjecture_holds"]:
        count_true += 1
    else:
        counterexample = trial_result["counterexample"]

mean_energy = Fraction(total_energy, len(results))
support_fraction = Fraction(count_true, len(results))

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_energy} std=0 support_fraction={support_fraction}")
elif support_fraction >= Fraction(4, 5): # At least 80% support
    print(f"RESULT: SUPPORTED mean={mean_energy} std=0 support_fraction={support_fraction}")
else:
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seeds[count_true-1]}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_edbab641.py", line 113, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_edbab641.py", line 70, in run_trial
    energy = additive_energy(truth_table)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_edbab641.py", line 54, in additive_energy
    if (truth_table[i][j] == truth_table[k][l]) != (truth_table[i][k] == truth_table[j][l]):
        ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError before producing results, preventing evaluation of support-/falsification | next: Fix the IndexError in additive_energy function by validating indices before access

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	87179
2	preregistration	ollama_remote	qwen3:8b	0	0	27340
3	novelty	ollama_remote	qwen3:8b	0	0	24225
4	novelty	ollama_remote	qwen3:8b	0	0	16286
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12945
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26730
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13699
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14560
9	judge	ollama_remote	qwen3:8b	0	0	112638

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 335600 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/249d7867db76.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/249d7867db76.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/249d7867db76.tar.gz` (if generated)